

SIMATS ENGINEERING
ASSESSMENT TOOL 1: Constraint-Based Problem Solving

COURSE NAME: COMPUTER VISION COURSE CODE: ITA0515 Slot B

COURSE OUTCOME COVERED

CO5: Develop computer vision applications using OpenCV for performing recognition and analysis on images and videos. (BTL 6)

Assessment Tool Weightage: 40%

QUESTIONS:5 Total Marks 50 Duration :60 Minutes Annexure A
Constraint-Based Problem Solving

Code of Conduct:

I V. Thanusri(192472265) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. IL= understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: V. Thanusri

Rubrics for Evaluation

Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)	Marks(100)
Problem Understanding	25%	Clearly defines the problem and identifies all relevant constraints accurately	Identifies most constraints with minor gaps	Partial understanding; misses key constraints	Fails to identify constraints correctly	
Constraint Analysis	25%	Analyzes constraints effectively and prioritizes them logically	Provides reasonable analysis with some prioritization	Limited analysis; weak prioritization	No meaningful analysis or prioritization	
Solution Development	25%	Develops optimal and feasible solutions satisfying all constraints	Develops workable solutions with minor limitations	Solutions partially satisfy constraints	Solutions do not meet constraints	
Application of Concepts	15%	Effectively applies appropriate concepts, methods, or tools	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply relevant concepts	

Justification	10%	Provides strong justification for decisions with logical reasoning	Provides justification with some clarity	Weak or unclear justification	No justification provided	
---------------	-----	--	--	-------------------------------	---------------------------	--

Constraint-Based Problem Solving: Analyze the problem and design an end-to-end computer vision pipeline using OpenCV, clearly identifying each stage from input acquisition to final output. Ensure that your solution satisfies all given constraints and justify your design decisions at each stage.

1. Real-Time Face Recognition System (Edge Deployment)

A company wants to deploy a face recognition system on low-power edge devices (e.g., Raspberry Pi).

The system must:

- A. Process live video streams in real time (<30 FPS latency)
- B. Work under varying lighting conditions
- C. Use limited memory and CPU resources
- D. Detect and recognize faces from a predefined dataset

Design a complete solution using OpenCV, specifying: image acquisition, preprocessing, feature detection, and recognition pipeline. Justify how your design satisfies all constraints.

2. Automated Video Surveillance with Alert System

A security firm needs a video surveillance system that:

- A. Detects motion and unusual activities
- B. Generates alerts in real time
- C. Works with standard CCTV video feeds
- D. Minimizes false positives in dynamic environments

Design an OpenCV-based system integrating video processing, feature detection, and motion analysis. Justify how your approach handles noise, dynamic backgrounds, and real-time constraints.

3. OCR System for Low-Quality Documents

A digitization company needs an OCR system that:

- A. Extracts text from low-resolution, noisy scanned images
- B. Handles varying fonts and illumination
- C. Works efficiently for batch processing

Design an image processing pipeline using OpenCV functions (enhancement, segmentation, etc.) for OCR. Justify how each stage improves recognition under given constraints.

4. Facial Expression Recognition for Mobile Application

A mobile app must detect facial expressions (happy, sad, neutral) with the following constraints:

- A. Limited processing power (mobile CPU)
- B. Real-time performance
- C. Robustness to slight head movement and lighting changes

Design a solution using OpenCV for face detection and expression analysis. Justify how your approach balances accuracy and efficiency.

5. Smart Traffic Monitoring System

A city authority needs a system to:

- A. Detect vehicles from live video streams
- B. Track movement across frames
- C. Count vehicles and display statistics
- D. Operate continuously with minimal delay

Design a complete OpenCV-based system including video handling, detection, tracking, and visualization (drawing functions). Justify how your system ensures accuracy and real-time performance.

Answers

1. Real-Time Face Recognition System (Edge Deployment)

Problem and objective

The objective is to build a lightweight face-recognition system for a low-power edge computer such as a Raspberry Pi. The system must accept live video, detect faces, identify them against a predefined database, operate under changing illumination, and maintain real-time performance with limited CPU, memory, and storage.

Overall architecture

A suitable pipeline is: Camera/CCTV input → frame acquisition → frame resizing and color conversion → illumination enhancement → face detection → face alignment/cropping → feature extraction → feature matching/classification → identity decision → display/logging. Processing should happen locally on the edge device so that video does not need to be continuously transmitted to a server.

1. Image acquisition

Use OpenCV's `cv2.VideoCapture()` to acquire frames from a USB camera, CSI camera, or compatible video stream. Capture at a moderate resolution such as 640×480 rather than processing full HD frames. The application can skip selected frames or process every second/third frame when the scene changes slowly. A fixed capture size reduces memory usage and gives predictable processing time.

2. Preprocessing

Convert each frame from BGR to grayscale when using a grayscale detector. Resize the frame to a smaller working resolution while preserving enough detail for face detection. To handle illumination changes, apply CLAHE (`cv2.createCLAHE()`) to improve local contrast. Mild Gaussian blur (`cv2.GaussianBlur()`) can suppress sensor noise, while excessive smoothing should be avoided because it can remove facial details.

Normalize the detected face crop before feature extraction.

3. Face/feature detection

For an edge implementation, a Haar Cascade classifier using `cv2.CascadeClassifier()` is simple and computationally inexpensive. The detector can use parameters such as `scaleFactor` and `minNeighbors` to control speed and false detections. A more robust alternative is OpenCV's DNN face detector using a lightweight model, if the Raspberry Pi has sufficient resources. Detection should be performed on a reduced-size frame, followed by mapping the bounding box back to the original frame if required.

4. Face alignment and feature extraction

After detecting a face, crop the bounding rectangle and resize it to a fixed size. If landmarks are available, align the face using the eye positions so that rotation and small head movements have less effect. For recognition, a lightweight embedding model is preferable to storing raw images. Each known person is represented by one or more feature vectors. The current face is converted to an embedding and compared with the stored vectors using cosine similarity or Euclidean distance.

5. Recognition

Create a predefined gallery containing names/IDs and their feature embeddings. For every detected face, calculate its embedding and compare it with the gallery. If the best similarity is above a selected threshold, display the corresponding identity; otherwise label the face as 'Unknown'. Multiple enrollment images per person can be averaged or stored separately to improve robustness to pose and lighting.

6. Real-time optimization

The main optimizations are resizing frames, processing only the region of interest where appropriate, avoiding repeated database loading, using lightweight models, limiting the number of detections per second, and separating frame capture from processing using a small queue or threaded pipeline. Recognition need not be performed on every frame: once a face is confidently identified, tracking can maintain its location between recognition updates.

7. Handling varying lighting

Lighting variation can be addressed using grayscale conversion, CLAHE, brightness/contrast normalization, and a diverse enrollment database. If a DNN-based embedding model is used, it should be

trained or selected for illumination robustness. Multiple images of each person under different lighting conditions also improve recognition reliability.

8. Resource and memory management

Use low-resolution frames, release unused camera and model resources with `cap.release()` and appropriate cleanup, and avoid storing full video in memory. Store only compact embeddings and identity metadata. Lightweight cascade or DNN models are more suitable for edge devices than large recognition networks. Quantized or CPU-optimized models can further reduce inference time.

9. Constraint evaluation

Constraint A is satisfied through reduced resolution, frame skipping, lightweight detection, and tracking. Constraint B is addressed by CLAHE, normalization, alignment, and varied enrollment images. Constraint C is satisfied by compact embeddings, lightweight models, and limited frame buffers. Constraint D is satisfied through a predefined gallery and nearest-neighbor/threshold-based matching.

Conclusion

The recommended design is a local OpenCV pipeline with VideoCapture, resizing and illumination correction, lightweight face detection, optional alignment, compact feature embeddings, and threshold-based matching. It provides a practical balance between recognition accuracy and Raspberry Pi resource limitations. A DNN detector/embedding model should be used when greater accuracy is required and the device can support the additional computation.

2. Automated Video Surveillance with Alert System

Problem and objective

The system must continuously monitor CCTV feeds, detect motion or unusual activity, generate real-time alerts, and avoid excessive false alarms caused by shadows, trees, rain, camera noise, and normal background movement.

Overall architecture

The proposed pipeline is: CCTV/RTSP or camera input → frame decoding → resizing → denoising → background/motion modeling → foreground segmentation → morphological filtering → contour/blob extraction → object tracking → activity analysis → event confirmation → alert generation and logging.

1. Video acquisition

Use `cv2.VideoCapture()` for USB cameras, files, or network streams where supported. Process frames at a controlled resolution such as 640×360 or 640×480. The application should monitor frame timestamps and recover from temporary stream failures. A separate capture thread can prevent slow network reads from blocking the processing pipeline.

2. Noise reduction

CCTV feeds may contain compression artifacts and sensor noise. Apply a small Gaussian blur or median filter before motion segmentation. Median filtering is especially useful for isolated noise, while Gaussian smoothing reduces high-frequency variations. The filter size must remain small so that object boundaries are not excessively blurred.

3. Motion detection

A background-subtraction method such as `cv2.createBackgroundSubtractorMOG2()` or KNN can create a foreground mask. MOG2 is useful when the background changes gradually. The learning rate should be selected carefully: a very high learning rate can absorb a moving object into the background, while a very low rate may fail to adapt to environmental changes.

4. Morphological processing

The foreground mask usually contains holes and isolated pixels. Apply `cv2.morphologyEx()` with opening to remove small noise and closing to fill small gaps. Dilation can strengthen connected foreground regions. This produces cleaner blobs and improves contour detection.

5. Feature/object detection

Use `cv2.findContours()` to locate foreground regions. Filter contours by area so that tiny noise regions are ignored. For more reliable surveillance, detected blobs can be classified using simple geometric features or a lightweight object detector so that people, cars, or other relevant objects are distinguished from shadows and environmental movement.

6. Tracking and unusual activity analysis

Track detected objects using centroid tracking, optical flow, or a lightweight tracker. For each object,

maintain its centroid, bounding box, velocity, direction, and time in the scene. Unusual activity can be defined using application-specific rules such as entering a restricted zone, crossing a virtual line, remaining stationary for too long, moving against the normal flow, or appearing during restricted hours.

7. False-positive reduction

Dynamic backgrounds are handled using adaptive background subtraction and careful parameter tuning. Shadows can be suppressed using shadow detection options in MOG2 or by checking object color/brightness characteristics. Area thresholds remove tiny moving regions. Temporal confirmation is important: an alert should be generated only when an event persists for several consecutive frames rather than from a single noisy frame.

8. Real-time alert generation

When an event is confirmed, the system can save a snapshot using `cv2.imwrite()`, record the event time and camera ID, and send a notification through the application's alert service. Alerts should be asynchronous so that sending a message does not stop video processing. A cooldown period can prevent repeated alerts for the same event.

9. Real-time optimization

Resize frames, process only relevant regions of interest, use efficient background subtraction, and avoid unnecessary per-pixel operations. Frame skipping can be used when computational load is high. A producer-consumer design can separate capture, processing, and alert delivery. The system should measure processing FPS and keep the processing queue short to avoid accumulating latency.

Conclusion

An OpenCV surveillance system based on adaptive background subtraction, morphological cleanup, contour filtering, tracking, and temporal event rules can satisfy the requirements. Noise is reduced through filtering, dynamic backgrounds are handled by adaptive models, false positives are reduced through spatial and temporal thresholds, and real-time operation is supported by low-resolution processing and asynchronous alert handling.

3. OCR System for Low-Quality Documents

Problem and objective

The objective is to extract readable text from low-resolution, noisy scanned documents with different

fonts and illumination conditions while processing many documents efficiently in batch mode. OCR accuracy depends strongly on the quality of the image supplied to the OCR engine, so OpenCV should prepare the image before recognition.

Overall architecture

The recommended pipeline is: document input → grayscale conversion → denoising → illumination correction → contrast enhancement → resizing → binarization → adaptive thresholding → morphological cleanup → skew detection/correction → text-region segmentation → OCR engine → post-processing and validation.

1. Image acquisition

Read scanned images with `cv2.imread()`. If color information is not necessary, convert using `cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)`. Grayscale reduces memory usage and simplifies subsequent thresholding. For batch processing, process files one at a time instead of loading the complete dataset into memory.

2. Denoising

Low-quality scans may contain speckles and compression noise. Use `cv2.medianBlur()` for salt-and-pepper noise or `cv2.GaussianBlur()` for general noise. The kernel should be small enough to preserve character strokes. Excessive smoothing can merge nearby letters and reduce OCR accuracy.

3. Illumination and contrast enhancement

Uneven illumination can make one part of a page bright and another dark. CLAHE can enhance local contrast without excessively amplifying the entire image. A background-estimation approach can also be used: estimate the slowly varying background using Gaussian blur and normalize the original grayscale image relative to that background. This makes text intensity more uniform across the page.

4. Rescaling

Small characters are difficult for OCR engines to recognize. Upscale the document using `cv2.resize()` with an appropriate interpolation method such as `INTER_CUBIC` or `INTER_LINEAR`. Upscaling does not create new information, but it can make character boundaries more suitable for subsequent thresholding and OCR.

5. Binarization and segmentation

Use `cv2.threshold()` with Otsu's method for relatively uniform pages. For uneven illumination, use `cv2.adaptiveThreshold()`, which computes a local threshold for different regions. Adaptive thresholding is particularly useful for old or poorly illuminated documents.

6. Morphological cleanup

Apply `cv2.morphologyEx()` to improve the binary image. Opening can remove small isolated noise, while closing can reconnect broken character strokes. A small rectangular or elliptical structuring element should be selected according to the character size. Morphological processing should be conservative to prevent neighboring characters from merging.

7. Skew correction

A rotated document causes incorrect text segmentation and reduces OCR accuracy. Detect dominant text-line orientation using edge detection and line analysis, then rotate the image by the estimated angle using an affine transformation. Straight text lines make OCR segmentation more reliable.

8. Text-region segmentation

Use contours, connected components, projection profiles, or morphology to identify text blocks and lines. Processing individual text regions can reduce background interference and allows the OCR engine to use an appropriate page segmentation mode. For documents with tables, headings, and multiple columns, region detection is especially important.

9. OCR and post-processing

The processed image can be passed to an OCR engine such as Tesseract. OpenCV handles image enhancement and segmentation while the OCR engine performs character recognition. Post-processing can correct common errors using dictionaries, expected character sets, regular expressions, or confidence thresholds. Low-confidence words can be flagged for review.

10. Batch-processing efficiency

For batch processing, use a file iterator and process one document at a time. Reuse OCR configuration and avoid keeping intermediate images in memory. Use multiprocessing only when CPU resources permit it. Save only the required output and optionally compressed intermediate results. This reduces memory consumption and increases throughput.

Conclusion

The combination of denoising, CLAHE, illumination correction, resizing, adaptive/Otsu thresholding, morphology, skew correction, segmentation, and OCR produces a robust pipeline for poor-quality documents. Each stage addresses a specific source of OCR error while batch-wise processing keeps memory usage manageable.

4. Facial Expression Recognition for Mobile Application

Problem and objective

The mobile application must classify expressions into happy, sad, and neutral while using limited CPU resources and maintaining real-time performance. The design should therefore use lightweight face detection, process only face regions, and use a compact expression classifier.

Overall architecture

The pipeline is: mobile camera → frame capture → downscaling → face detection → face crop → alignment/normalization → lightweight expression feature extraction/classification → temporal smoothing → display result. OpenCV performs camera handling, preprocessing, detection, and tracking; a compact classifier can perform expression classification.

1. Camera acquisition

Use OpenCV camera capture where supported by the mobile deployment environment, or integrate the platform camera API and pass frames to OpenCV. Process a moderate resolution rather than full-resolution frames. This reduces memory bandwidth and CPU consumption.

2. Face detection

A lightweight Haar Cascade can provide fast detection on CPU. A compact OpenCV DNN face detector is an alternative when greater robustness is required. Detection does not need to run on every frame. Once a face is detected, tracking can estimate its location for subsequent frames.

3. Face preprocessing

Crop the detected face and resize it to a fixed classifier input size. Convert to grayscale when the classifier does not require color. Apply histogram equalization or CLAHE to reduce sensitivity to illumination changes. Normalize pixel intensity so that the classifier receives a more consistent input.

4. Handling slight head movement

Use facial landmarks if available to locate the eyes and mouth, then align the face. Alternatively, a tracker can follow the face bounding box between detections. Small changes in scale and position can be handled by resizing the crop. Training the expression classifier with slightly rotated and translated examples also improves robustness.

5. Expression feature extraction

For a lightweight classical solution, extract features such as Local Binary Patterns (LBP), HOG descriptors, or normalized pixel features from the face region. HOG captures shape and edge information, while LBP is useful for local texture patterns. A small SVM or other compact classifier can then classify happy, sad, and neutral expressions.

6. Alternative lightweight DNN

If a small trained neural network is available, use a compact CNN with a low input resolution. The network can provide better expression accuracy than handcrafted features, while quantization and CPU optimization help maintain mobile performance. OpenCV's DNN module can perform inference without requiring a large deep-learning runtime.

7. Temporal smoothing

Expressions can fluctuate from frame to frame because of noise or partial occlusion. Store the predictions from a short sequence of frames and select the majority class or use an exponentially smoothed confidence score. This improves visual stability without significantly increasing computation.

8. Lighting robustness

CLAHE, grayscale normalization, and training with varied lighting conditions reduce sensitivity to brightness changes. Avoid strong global thresholding because expression classification requires subtle facial information. A diverse training set is especially important for mobile use.

9. Efficiency measures

Run face detection periodically, track between detections, resize frames, classify only the face ROI, and avoid unnecessary color conversions. Keep only a short temporal buffer. These techniques reduce CPU usage and improve FPS while preserving useful expression information.

Conclusion

A practical mobile solution combines OpenCV camera processing, lightweight face detection, face alignment, illumination normalization, a compact HOG/LBP or small CNN classifier, and temporal smoothing. The system balances accuracy and efficiency by avoiding expensive processing on the full image and performing expression analysis only on the detected face.

5. Smart Traffic Monitoring System

Problem and objective

The goal is to continuously detect vehicles from live video, track their movement across frames, count vehicles, and display useful statistics with minimal delay. The system should remain accurate despite changes in vehicle position, scale, and moderate traffic density.

Overall architecture

The proposed pipeline is: live CCTV/video input → frame acquisition → resizing and ROI selection → vehicle detection → object association/tracking → line or zone crossing → counting → statistics calculation → visualization using OpenCV drawing functions → display/logging.

1. Video handling

Use `cv2.VideoCapture()` to read a camera, CCTV stream, or video file. Verify that the stream opened correctly and handle dropped frames or temporary failures. Resize frames to a suitable processing resolution. A region of interest can exclude buildings, sky, sidewalks, or other areas where vehicles are not expected.

2. Vehicle detection

There are two practical approaches. For a fixed camera with a stable background, background subtraction using `cv2.createBackgroundSubtractorMOG2()` can identify moving vehicles. For higher accuracy in crowded or changing scenes, use a lightweight object detector through OpenCV's DNN module and retain vehicle classes such as car, bus, truck, and motorcycle. A detector gives class labels and bounding boxes.

3. Preprocessing and noise reduction

For background-subtraction systems, apply a small blur before subtraction and use morphological

opening and closing to clean the foreground mask. Filter contours by area to eliminate small noise. For DNN detection, resize and normalize frames according to the model's input requirements.

4. Tracking

Detection alone does not establish which vehicle in one frame corresponds to a vehicle in the next. Maintain a track ID for each vehicle using centroid tracking, optical flow, or a lightweight tracker. For each tracked vehicle, store its bounding box, centroid, last-seen frame, movement direction, and class. Track IDs prevent the same vehicle from being counted repeatedly.

5. Vehicle counting

Define a virtual counting line using two points or a counting zone. When a tracked vehicle's centroid crosses the line in the required direction, increment the appropriate vehicle count and mark that track as already counted. Separate lines or zones can be used for incoming and outgoing traffic. This provides counts by direction and vehicle type.

6. Statistics

Maintain counters such as total vehicles, cars, buses, trucks, motorcycles, incoming vehicles, and outgoing vehicles. Average traffic flow can be calculated over fixed time intervals. If timestamps are available, vehicles per minute or vehicles per hour can be displayed.

7. Visualization

OpenCV drawing functions make the system easy to interpret. Use `cv2.rectangle()` for vehicle bounding boxes, `cv2.circle()` for centroids, `cv2.line()` for the counting line, and `cv2.putText()` for IDs, class names, FPS, and counters. The display should clearly distinguish tracked vehicles from the counting region.

8. Accuracy considerations

Use an appropriate confidence threshold to reject weak detections. Non-maximum suppression removes duplicate bounding boxes. Restricting detection to a relevant ROI reduces false detections. Tracking stabilizes object identity across frames. For fixed cameras, background models should be updated gradually to handle environmental changes. For difficult scenes, a trained DNN detector is more reliable than simple background subtraction.

9. Real-time performance

Use moderate input resolution, detect periodically when tracking is reliable, process only the ROI, use

lightweight models, and avoid expensive operations on the entire frame. A separate capture thread can keep the camera responsive. FPS should be measured using frame timestamps, and the system should avoid building a long processing queue because queue growth increases delay.

10. Continuous operation and reliability

The application should release resources correctly, recover from camera disconnections, log counts periodically, and prevent duplicate counting. Memory should be bounded by removing tracks that have not been observed for a specified number of frames. Long-running systems should also monitor FPS and processing latency.

Conclusion

A complete traffic-monitoring solution can combine OpenCV video capture, ROI-based vehicle detection, lightweight tracking, virtual-line counting, statistics, and visualization. Background subtraction is suitable for simple fixed-camera scenes, while an OpenCV DNN detector provides better robustness in complex traffic. Tracking prevents duplicate counts, and resolution/model optimization maintains continuous real-time operation.

Summary of Recommended Techniques

- Q1: VideoCapture + resizing + CLAHE + lightweight face detector + alignment + face embeddings + threshold matching + tracking.
- Q2: VideoCapture + denoising + MOG2/KNN background subtraction + morphology + contours + tracking + temporal event rules + asynchronous alerts.
- Q3: Grayscale + denoising + CLAHE/illumination correction + resizing + Otsu/adaptive thresholding + morphology + skew correction + segmentation + OCR.
- Q4: Camera input + lightweight face detector + face ROI preprocessing + LBP/HOG or compact CNN + tracking + temporal smoothing.
- Q5: VideoCapture + ROI + MOG2 or lightweight DNN vehicle detection + tracking + virtual line crossing + counting + statistics + drawing functions.